VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING




# Deep Learning and Its Applications (CO3133)

## ASSIGNMENT-1

| | | | |
|---|---|---|---|
| **Lecturer** | Le Thanh Sach | | |
| **Class** | CC01 | | |
| **Team** | G-09 | | |
| **Students** | Nguyen Vu Long | 2352696 | long.nguyenty@hcmut.edu.vn |
| | Le Phuoc Vu | 2353341 | vu.lephuocvu13@hcmut.edu.vn |
| | Nguyen Hoang Quoc | 2353027 | quoc.nguyenhoang2305@hcmut.edu.vn |

**Ho Chi Minh, 10/2026**

# TABLE OF CONTENTS

**1 Introduction** **2**
1.1 Background and Motivation . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives and Scope . . . 2
1.4 Proposed Approach . . . 2

**2 Dataset and Exploratory Analysis** **3**
2.1 Dataset Description . . . 3
2.2 Data Preparation and Splitting . . . 3
2.3 Exploratory Data Analysis . . . 4
2.4 Preprocessing . . . 8

**3 Methodology** **11**
3.1 Shared data and workflow . . . 11
3.2 Pixel-based baselines . . . 12
3.3 Models for spatial and sequential structure . . . 12
3.4 Transformer . . . 13
3.5 Learning objective . . . 14
3.6 Training procedure . . . 14
3.7 Evaluation measures . . . 15
3.8 Runtime, reproducibility and comparison scope . . . 15

**4 Experimental Results** **16**
4.1 Overall classification performance . . . 16
4.2 Learning behavior and checkpoint selection . . . 17
4.3 Model size and computational cost . . . 19

**5 Error Analysis** **19**
5.1 Performance by class . . . 19
5.2 Confusion patterns in the CNN . . . 20
5.3 Individual predictions and shared errors . . . 21

**6 Discussion and Limitations** **22**
6.1 What the comparison shows . . . 22
6.2 Limitations . . . 23
6.3 Next experiments . . . 23

**7 Conclusion** **24**

**A Detailed Class Results** **26**

**B Additional Confusion Matrices** **26**

**C Reproducibility and Evidence** **27**
C.1 Environment and saved files . . . 27
C.2 Commands to reproduce the experiment . . . 28
C.3 Checks performed for this report . . . 28

# 1 Introduction

## 1.1 Background and Motivation

Image classification maps an image to a category. In an online product catalogue, for example, a deep learning classifier could help sort images into the correct category. The classification is only one part of the system. The reliability of the result also depends on the dataset, the representation of each image, the separation between training and evaluation data and also the way model quality and computational cost are measured.

Fashion-MNIST provides a practical benchmark for investigating these factors. Its compact grayscale images allow different deep learning architectures and configurations to be evaluated efficiently, while visually similar categories introduce meaningful classification challenges.

This project investigates a range of machine learning and deep learning approaches, from simple baseline models to more advanced architectures. By comparing their predictive performance, generalization ability, training behavior, and computational requirements under a consistent experimental setup, we aim to understand the strengths, limitations, and trade-offs of different approaches to image classification.

## 1.2 Problem Statement

The task is a supervised, single-label, ten-class image classification. Let $X \in \{0, \dots, 255\}^{28\times 28}$ be a raw grayscale image and $y \in \{0, \dots, 9\}$ its label. A classifier with parameters $\theta$ receives a preprocessed representation $T(X)$ and produces ten real-valued class scores, or logits:

$$f_\theta(T(X)) = \boldsymbol{z} \in \mathbb{R}^{10}, \qquad \widehat{y} = \underset{k\in\{0,\dots,9\}}{\arg\max}\ z_k. \tag{1}$$

The task focuses on assigning a single category to each image. We use this classification problem to compare how different neural network architectures learn and represent image features.

## 1.3 Objectives and Scope

The objective of this project is to build an end-to-end deep learning pipeline for image classification using the Fashion-MNIST dataset. Through this project, we aim to understand how different neural network architectures process image data and how their designs influence classification performance. We implement and compare five architectures: Linear Classifier, Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), LSTM, and Transformer.

The scope of this project covers the complete workflow, from data exploration and preprocessing to model training and evaluation using PyTorch. All models are trained and evaluated on the same dataset splits to ensure a fair comparison. We analyze their performance using classification accuracy, macro-F1 score, training time, inference time, and model complexity. In addition, we examine training behavior and prediction errors to better understand the strengths and limitations of each architecture.

## 1.4 Proposed Approach

We use a common experimental pipeline to compare the five architectures. Fashion-MNIST is divided into fixed training, validation, and test subsets, with normalization statistics computed only from the training subset. Each image is represented as a flattened vector for the Linear Classifier and MLP, a two-dimensional grid for the CNN, a sequence of rows for the LSTM, or a sequence of patches for the Transformer. Each model is trained using a shared optimization protocol, and its best checkpoint is selected by the lowest validation loss. The selected models are then evaluated on the test set to compare classification performance, computational cost, and prediction errors.

# 2 Dataset and Exploratory Analysis

## 2.1 Dataset Description

Fashion-MNIST contains grayscale images of clothing and accessories from ten categories. It provides 60,000 official training images and 10,000 test images. Each image is a $28 \times 28$ array of unsigned 8-bit pixel values ranging from 0 to 255, with one integer class label. Table 1 lists the category names used in this report.

| **Label** | **Category** | **Label** | **Category** |
|---|---|---|---|
| 0 | T-shirt/top | 5 | Sandal |
| 1 | Trouser | 6 | Shirt |
| 2 | Pullover | 7 | Sneaker |
| 3 | Dress | 8 | Bag |
| 4 | Coat | 9 | Ankle boot |

Table 1: Fashion-MNIST class labels

The dataset includes several visually similar categories, such as T-shirt/top and Shirt, or Pullover and Coat, which makes classification more challenging. Despite the small image size, models must learn to distinguish subtle differences in shape and texture to correctly identify each category.

Fashion-MNIST is suitable for this project because its standardized image format allows different neural network architectures to be compared under consistent conditions. However, its low-resolution grayscale images do not fully represent real-world product images, which often contain colors, complex backgrounds, and greater visual variation.

## 2.2 Data Preparation and Splitting

The official Fashion-MNIST training dataset is divided into training and validation subsets using a stratified 90/10 split with a fixed random seed of 42. This produces 54,000 training images and 6,000 validation images, while the official 10,000-image test set remains separate. Stratification ensures that all ten classes maintain equal representation across the partitions, as shown in Table 2.

| Partition | Images | Per class | Role |
|---|---|---|---|
| Training | 54,000 | 5,400 | Fit normalization and update model parameters. |
| Validation | 6,000 | 600 | Select the lowest-validation-loss checkpoint. |
| Official test | 10,000 | 1,000 | Evaluate the fixed selected checkpoint. |

Table 2: Dataset partitions

The training set is used to compute normalization statistics and update model parameters, while the validation set is used to monitor training performance and select the best checkpoint based on validation loss. The test set is reserved for the final evaluation. The following diagram illustrates the data preparation and evaluation process.

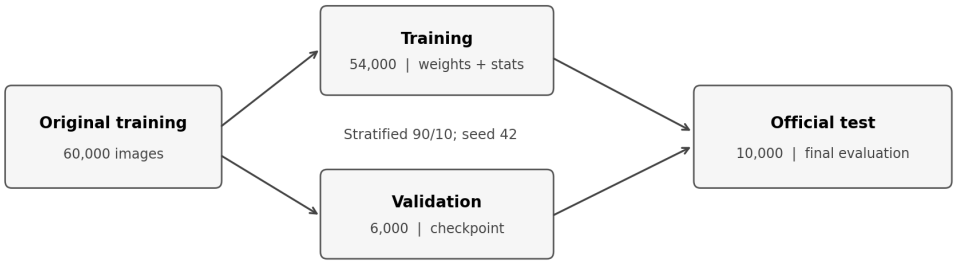



Figure 1: Data split and evaluation

To ensure reproducibility and prevent data leakage, the training and validation indices are fixed and verified to contain no overlapping samples. An additional integrity check is performed using SHA-256 hashes to identify exact duplicate images within and across partitions. As shown in Table 3, no exact duplicates are detected. However, this method cannot identify visually similar images or different images of the same product.

| Duplicate Check | Count |
|---|---|
| Duplicate images within training set | 0 |
| Duplicate images within validation set | 0 |
| Duplicate images within test set | 0 |
| Shared images between training and validation | 0 |
| Shared images between training and test | 0 |
| Shared images between validation and test | 0 |

Table 3: Duplicate image audit

## 2.3 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed on the training dataset to better understand its characteristics before model training. The analysis focuses on class distribution, image variation, pixel intensity, and similarities between categories.

### 2.3.1 Class Distribution.

The dataset is evenly distributed across all ten categories, with 5,400 training, 600 validation, and 1,000 test images per class, as shown in Figure 2. This balanced distribution helps prevent models from favoring certain classes due to differences in sample counts. Therefore, no additional class balancing techniques are required.

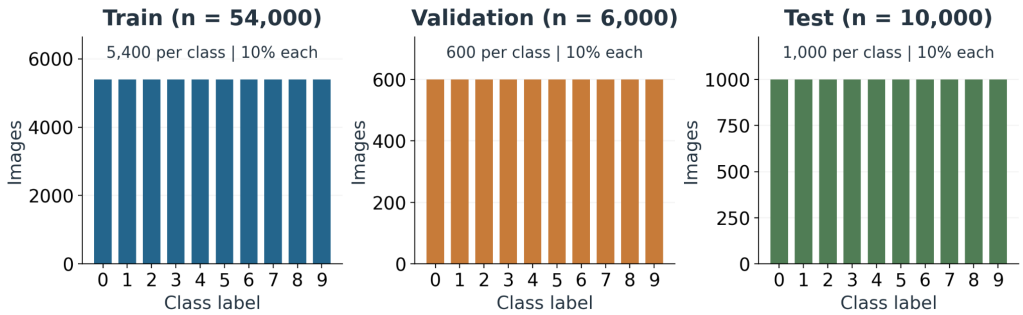



Figure 2: Class distribution

### 2.3.2 Individual Image Variation

Figure 3 presents three randomly selected training images from each of the ten categories using a fixed random seed of 42. The examples highlight variations in shape, brightness, and appearance both within and across categories. Some categories, such as Trouser and Bag, have distinctive shapes that make them visually recognizable. In contrast, upper-body garments such as T-shirt/top, Shirt, Pullover, and Coat share similar features, making them more difficult to distinguish. These observations suggest that classification models need to learn detailed visual features rather than relying only on general shapes to accurately identify each category.

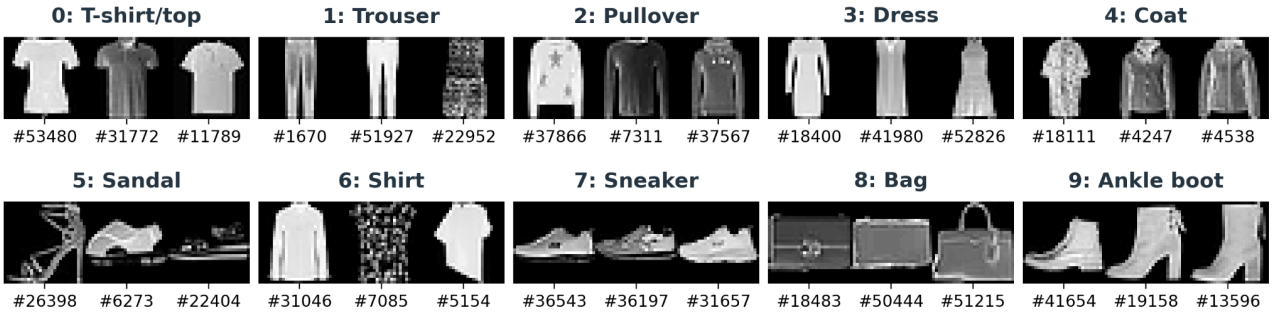



Figure 3: Training image samples

### 2.3.3 Pixel intensity and background

The training dataset contains a total of 42,336,000 pixels, with a mean intensity of 72.98 and a standard deviation of 90.06. Approximately 50.21% of the pixels have an intensity value of zero, indicating that a large portion of the images consists of dark regions. However, none of the training images is entirely blank.

Figure 4 illustrates the pixel intensity distribution. The left panel shows the distribution of all pixel values, highlighting the large proportion of zero-valued pixels. The right panel focuses on nonzero pixels to provide a clearer view of intensity variation.

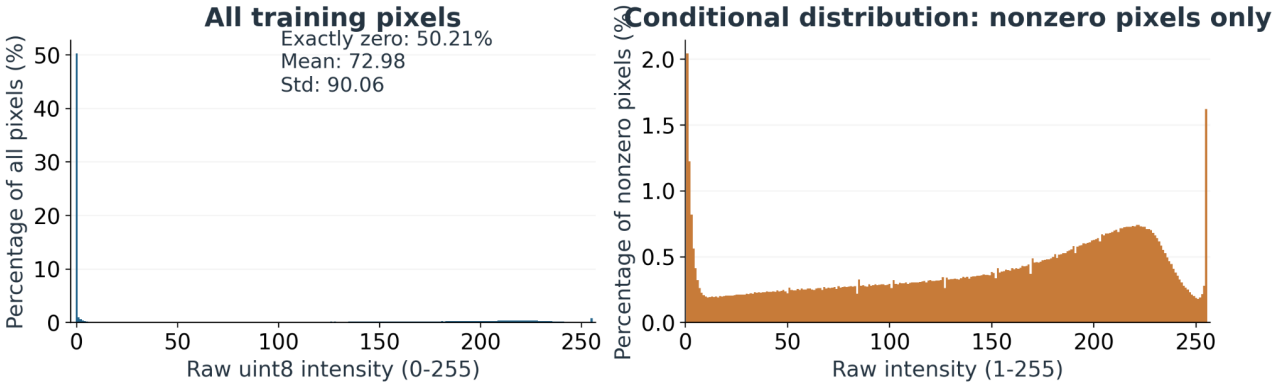



Figure 4: Pixel intensity distribution

### 2.3.4 Brightness and Pixel Coverage

Image brightness is calculated as the average intensity of all 784 pixels, ranging from 0 to 255. Nonzero pixel coverage represents the percentage of pixels with intensity greater than zero, providing an approximate measure of how much of an image contains visible information.

Figure 5 illustrates the distributions of brightness and pixel coverage across the ten categories, while Table 4 summarizes their statistics. Coat has the highest mean brightness (98.15), whereas Sandal has the lowest (34.87). In terms of pixel coverage, Pullover has the highest average (64.84%), while Sandal has the lowest (32.10%).

Although these characteristics differ across categories, their distributions overlap considerably. This suggests that brightness and pixel coverage alone are insufficient to distinguish all classes, highlighting the importance of learning more detailed visual features for accurate classification.

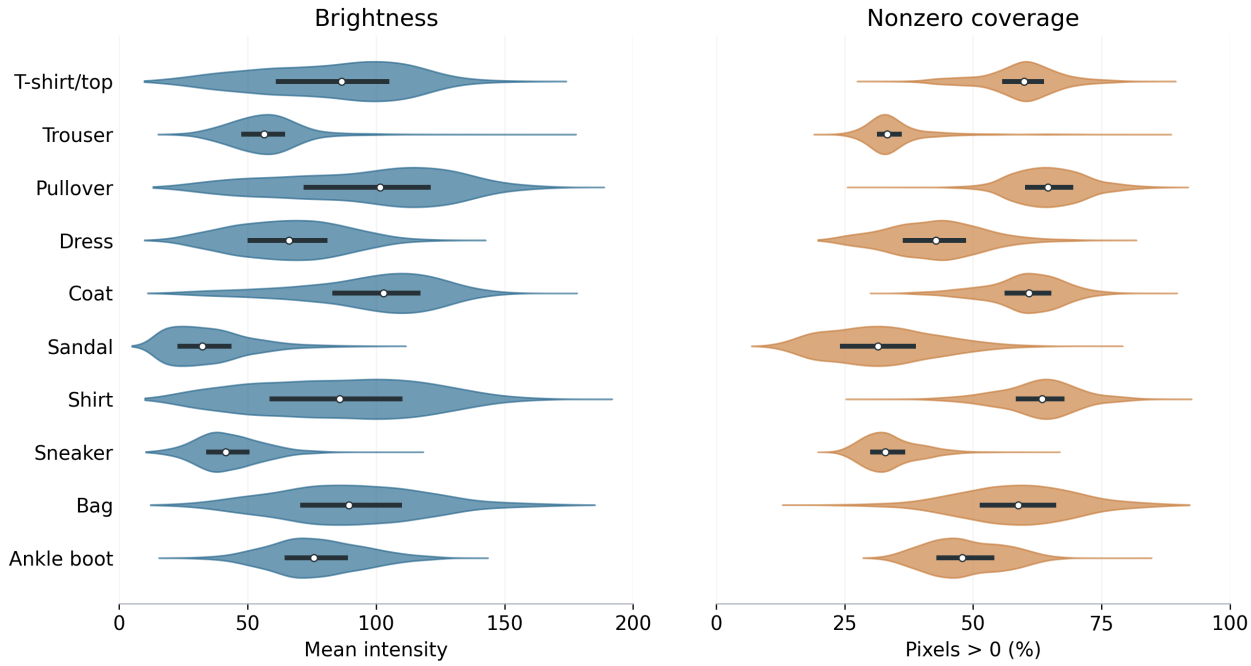



Figure 5: Brightness and pixel coverage

| Label | Class | Mean Brightness | Brightness SD | Mean Nonzero (%) |
|---|---|---|---|---|
| 0 | T-shirt/top | 83.02 | 28.84 | 59.38 |
| 1 | Trouser | 56.91 | 14.55 | 34.91 |
| 2 | Pullover | 96.17 | 32.70 | 64.84 |
| 3 | Dress | 65.91 | 21.33 | 42.67 |
| 4 | Coat | 98.15 | 26.46 | 60.19 |
| 5 | Sandal | 34.87 | 15.74 | 32.10 |
| 6 | Shirt | 84.84 | 32.90 | 62.87 |
| 7 | Sneaker | 42.81 | 12.85 | 33.81 |
| 8 | Bag | 90.28 | 28.81 | 58.61 |
| 9 | Ankle boot | 76.87 | 18.32 | 48.48 |

Table 4: Class brightness and coverage

### 2.3.5 Spatial Class Averages and Variability

To examine the spatial characteristics of each class, we compute the mean image and pixel-wise standard deviation from its 5,400 training samples. The mean image represents the typical class appearance, while the standard deviation indicates variations in pixel intensity.

For each class $k$, these statistics are defined as:

$$
\begin{aligned}
M_{k,r,c} &= \frac{1}{n_k} \sum_{i \in I_k} U_{i,r,c}, \\
S_{k,r,c} &= \sqrt{\frac{1}{n_k} \sum_{i \in I_k} (U_{i,r,c} - M_{k,r,c})^2}.
\end{aligned}
\tag{2}
$$

*where*
$M_k$ : class mean image,
$S_k$ : class standard deviation map,
$U_i$ : normalized image ($X_i/255$),
$I_k$ : set of samples in class $k$,
$n_k$ : 5,400 samples per class,
$(r, c)$ : pixel position.

Figure 6 shows that mean images preserve recognizable class shapes, although some categories, such as pullovers and coats, appear similar. Higher standard deviations occur mainly around object boundaries, reflecting differences in shape and positioning. These patterns highlight the challenges of within-class variation and visual similarity between categories.

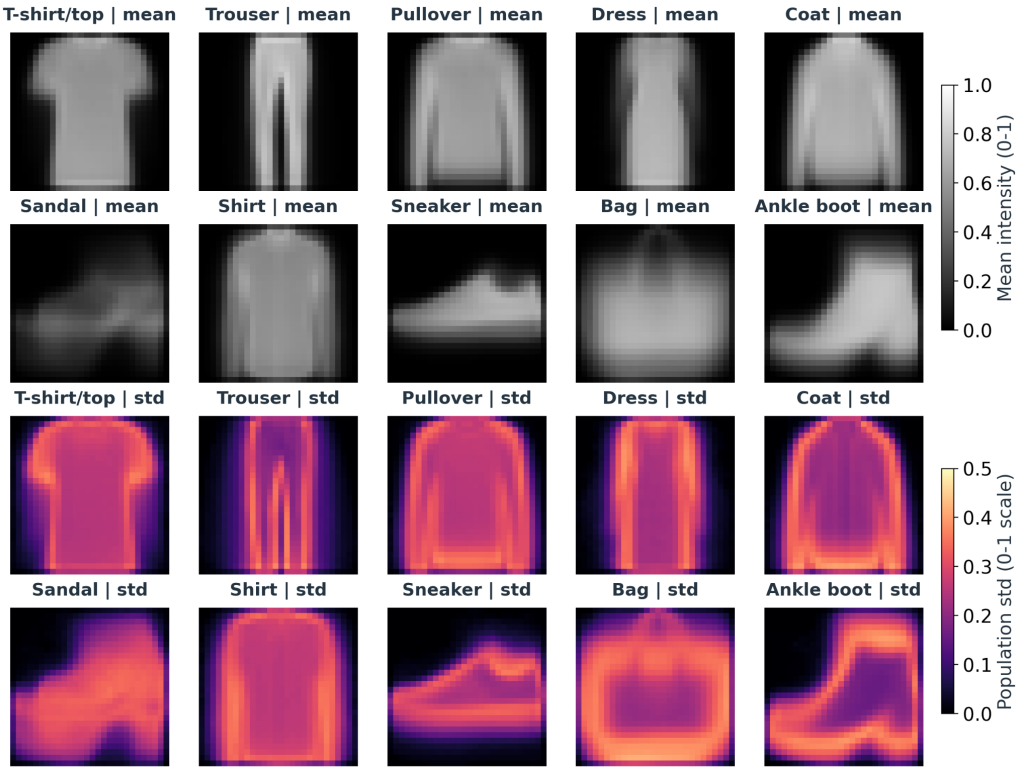



Figure 6: Class means and variability

### 2.3.6 Similarity Between Class Prototypes

To quantify similarities between clothing categories, each class mean image is flattened into a 784-dimensional vector. Pairwise cosine similarity is then calculated as:

$$
s_{k\ell} = \frac{\mathbf{m}_k^\top \mathbf{m}_\ell}{\|\mathbf{m}_k\|_2 \|\mathbf{m}_\ell\|_2}, \qquad \mathbf{m}_k = \mathrm{vec}(M_k) \in \mathbb{R}^{784}. \tag{3}
$$

Figure 7 shows the cosine similarity between class-average images. The highest similarity between distinct classes is 0.9873, observed between Coat and Shirt, indicating that their average images share highly similar spatial patterns.

These results highlight potential challenges in distinguishing visually similar categories. However, cosine similarity only measures resemblance between class averages and does not account for individual image variations or model predictions. Therefore, the observed similarities will be compared with model confusion matrices during the evaluation stage.

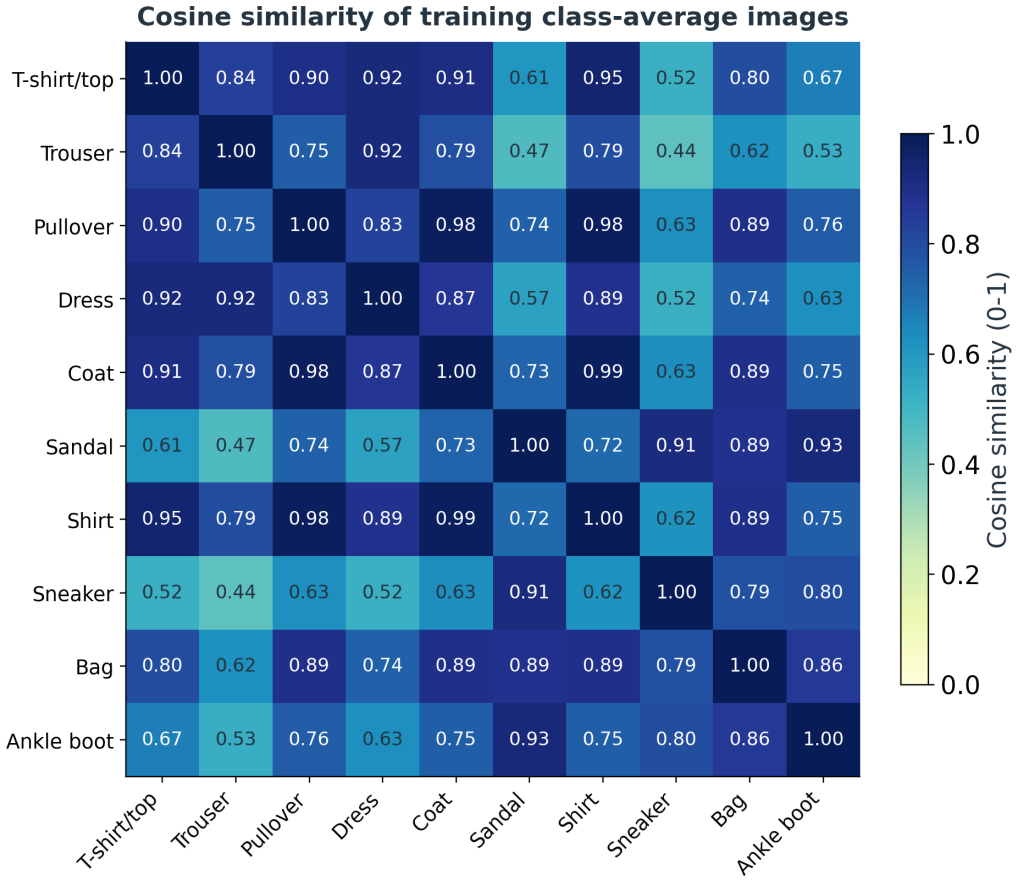



Figure 7: Class prototype similarity

## 2.4 Preprocessing

After exploring the dataset, a consistent preprocessing pipeline is applied to prepare the images for model training. Although Fashion-MNIST images share the same resolution and grayscale format, their pixel intensities and visual characteristics vary across categories.

The preprocessing process consists of converting images into tensors, scaling and standardizing pixel intensities, and arranging the resulting data into suitable input representations for each neural network architecture. The same numerical transformation is used for all five models to ensure a consistent experimental setup.

### 2.4.1 Image Conversion and Intensity Scaling

Each Fashion-MNIST image is originally represented as a $28 \times 28$ array of unsigned 8-bit integers, with pixel intensities ranging from 0 to 255. Before training, the images are converted into 32-bit floating-point tensors using the `ToTensor` transformation in PyTorch.

This conversion adds a single channel dimension and scales the original pixel intensities to the range $[0, 1]$:

$$U_{i,0,r,c} = \frac{X_{i,r,c}}{255}, \qquad U_i \in [0,1]^{1\times 28\times 28}. \tag{4}$$

*where* $X_i$ : original image,
$U_i$ : scaled image,
$(r, c)$ : pixel coordinates.

Scaling places all pixel values within a consistent numerical range, making them suitable for neural network optimization. The transformation preserves the original image dimensions and relative intensity differences. No resizing or interpolation is required because all images already have the same resolution.

### 2.4.2 Standardization and Data Leakage Prevention

After scaling, the pixel intensities are standardized using the mean and standard deviation calculated from the training dataset. Unlike intensity scaling, standardization centers the data around the training distribution and expresses pixel values relative to its variability.

The mean $\mu$ and population standard deviation $\sigma$ are calculated across all training pixels:

$$\begin{aligned} \mu &= \frac{1}{N} \sum_{i\in I_{\text{train}}} \sum_{r=0}^{27} \sum_{c=0}^{27} U_{i,0,r,c}, \\ \sigma &= \sqrt{\frac{1}{N} \sum_{i\in I_{\text{train}}} \sum_{r=0}^{27} \sum_{c=0}^{27} (U_{i,0,r,c} - \mu)^2}. \end{aligned} \tag{5}$$

*where* $I_{\text{train}}$ : set of 54,000 training images,
$N$ : total training pixels $(42,336,000)$,
$\mu$ : training pixel mean,
$\sigma$ : training pixel standard deviation.

The calculated statistics are $\mu = 0.286209$ and $\sigma = 0.353160$. Each scaled image is then standardized as follows:

$$Z_{i,0,r,c} = \frac{U_{i,0,r,c} - \mu}{\sigma}. \tag{6}$$

*where* $Z_i$ denotes the standardized image.

Standardization changes the numerical scale without removing image information. For example, original intensities of 0 and 255 correspond approximately to standardized values of $-0.810$ and 2.021, respectively. Negative values are expected and are not clipped.

To prevent data leakage, the normalization statistics are calculated exclusively from the training partition. The validation and test datasets use the same precomputed mean and standard deviation without recalculation. This ensures that information from held-out samples does not influence the preprocessing parameters used during training.

### 2.4.3 Input Representations for Different Models

Although all five architectures receive the same standardized pixel values, their input structures differ according to how each model processes image information. The preprocessing pipeline therefore reshapes the standardized images into the required tensor formats.

Table 5 summarizes the input representations for each architecture.

Here, $B$ represents the batch size, which is set to 128 in the shared training configuration.

| Model | Input Shape | Representation |
|---|---|---|
| Linear / MLP | $(B, 784)$ | Flattened pixel vector |
| CNN | $(B, 1, 28, 28)$ | Original image grid |
| LSTM | $(B, 28, 28)$ | 28 sequential rows with 28 features |
| Transformer | $(B, 49, 16)$ | 49 non-overlapping $4 \times 4$ patches |

Table 5: Model input shapes

**2.4.3.1 Linear Classifier and MLP.** For the Linear Classifier and Multilayer Perceptron, each standardized image is flattened into a vector containing 784 pixel values. The pixels are arranged in row-major order, preserving their original values and positions within the vector.

This representation allows both models to process the entire image as a single feature vector. However, the input does not explicitly preserve the two-dimensional neighborhood structure used by convolutional operations.

**2.4.3.2 Convolutional Neural Network.** The CNN receives the standardized image in its original spatial format $(1, 28, 28)$. Preserving the channel, row, and column dimensions allows convolutional filters to learn local visual patterns from neighboring pixels.

No handcrafted features or additional image channels are introduced during preprocessing.

**2.4.3.3 LSTM.** For the LSTM, each image is represented as a sequence of 28 rows, where each row contains 28 pixel features. The rows are processed in order from the top to the bottom of the image.

This representation allows the recurrent network to examine spatial information sequentially. The sequence describes ordered positions within a static image rather than temporal observations.

**2.4.3.4 Transformer.** For the Transformer, each image is divided into non-overlapping $4 \times 4$ patches. Since the original resolution is $28 \times 28$, this produces a $7 \times 7$ grid containing 49 patches.

Each patch is flattened into a vector of 16 pixel values, resulting in an input tensor of shape $(49, 16)$ for each image. Patches are ordered from left to right and top to bottom.

The patch extraction can be expressed as:

$$P_{i,7a+b,4u+v} = Z_{i,0,4a+u,4b+v}, \tag{7}$$

*where* $P_i$ : patch representation of image $i$,
$a, b \in \{0, \ldots, 6\}$ : patch coordinates,
$u, v \in \{0, \ldots, 3\}$ : pixel coordinates within a patch.

This produces $49 \times 16 = 784$ values per image, without padding or discarding pixels. The subsequent patch embedding and positional encoding belong to the Transformer architecture rather than the preprocessing stage.

All five representations preserve the same 784 standardized pixel values. Figure 8 illustrates how an identical image is rearranged into the different input formats. Consequently, differences in model performance can be studied under a common input information baseline.

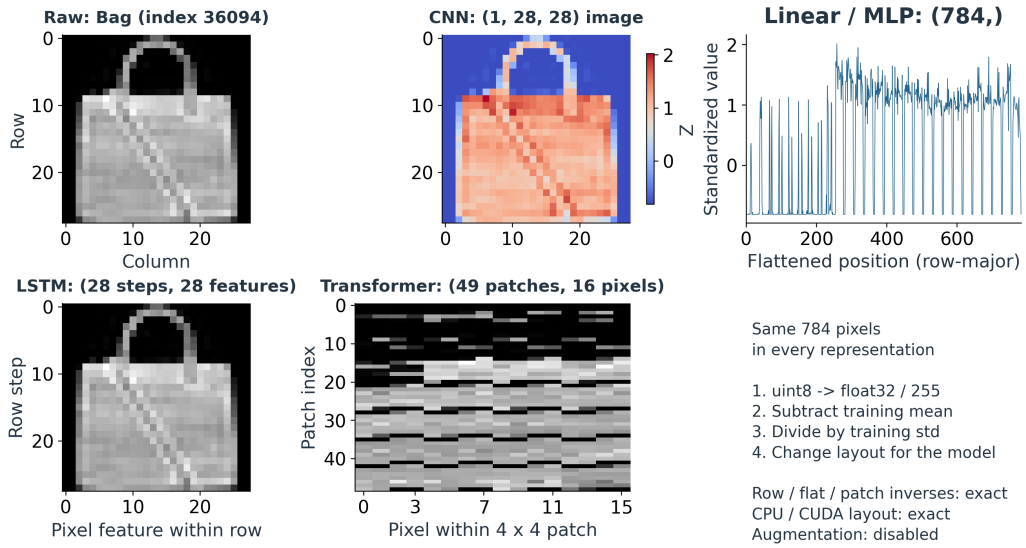



Figure 8: Input representations by model

### 2.4.4 Data Augmentation and Preprocessing Verification

To maintain a consistent baseline, no additional image augmentation is applied during training. Operations such as random rotations, horizontal flips, resizing, denoising, and binarization are excluded from the shared preprocessing configuration.

The EDA also shows that no training image is entirely blank and no exact duplicates are found in the checked dataset partitions. Therefore, no samples are removed based on brightness, pixel coverage, or duplicate detection.

The preprocessing pipeline is verified through numerical and structural consistency checks. The standardized training data has a mean approximately equal to zero and a population standard deviation approximately equal to one, with deviations below $10^{-6}$.

Additional checks confirm that flattening, row-sequence conversion, and patch extraction preserve the original pixel values and ordering. Reconstructing an image from its extracted patches also confirms that no pixels are lost during the representation process.

Finally, preprocessing results are checked for consistency between CPU and CUDA operations. These verification steps ensure that all five architectures receive equivalent image information and establish a reproducible foundation for the subsequent training and evaluation stages.

# 3 Methodology

We compare five models under the same data split and training settings: a Linear classifier, MLP, CNN, LSTM and Transformer. Their main difference is how they use the image. The first two operate on flattened pixels, the CNN keeps the image grid, the LSTM reads it row by row, and the Transformer processes small patches. The models and evaluation procedure are described below.

## 3.1 Shared data and workflow

Fashion-MNIST contains ten classes of $28 \times 28$ grayscale images [1]. As described in Section 2, the official training set is divided into 54,000 training images and 6,000 validation images using a stratified split with seed 42. The official test set contains 10,000 images. Training fits the model weights, validation chooses which saved weights to keep, and testing measures the selected model.

All models use the preprocessing from Section 2. Pixel values are divided by 255 and standardized with the training mean and standard deviation, approximately 0.286209 and 0.353160. The same constants are reused

for validation and testing. No images are resized, augmented or removed.

Table 6 gives the input to each model. Here, $B$ is the number of images in a batch. Each representation contains all 784 pixel values per image. The labels are integer class indices from 0 to 9. Every model returns a tensor of shape $(B, 10)$ containing raw class scores, called logits.

| Model | Input shape | Arrangement | Parameters |
|---|---|---|---|
| Linear | $(B, 784)$ | One vector per image | 7,850 |
| MLP | $(B, 784)$ | One vector per image | 235,146 |
| CNN | $(B, 1, 28, 28)$ | One grayscale image grid | 421,642 |
| LSTM | $(B, 28, 28)$ | 28 rows, each with 28 pixels | 82,186 |
| Transformer | $(B, 49, 16)$ | 49 patches, each with 16 pixels | 71,946 |

Table 6: Model inputs and parameter counts

## 3.2 Pixel-based baselines

### 3.2.1 Linear classifier

The Linear model assigns a weight to each pixel for each class. For a flattened image $x \in \mathbb{R}^{784}$, its output is

$$z = Wx + b, \qquad W \in \mathbb{R}^{10\times 784}, \quad b \in \mathbb{R}^{10}.$$

The largest score determines the prediction. This model provides a small baseline with no hidden layer. It can learn which pixel positions support a class, but cannot learn nonlinear combinations of those pixels.

### 3.2.2 Multilayer perceptron

The MLP adds two hidden layers with 256 and 128 units. Each hidden layer is followed by ReLU and dropout with probability 0.2. ReLU sets negative values to zero, while dropout randomly suppresses some activations during training.

$$784 \longrightarrow 256 \longrightarrow 128 \longrightarrow 10.$$

The hidden layers allow the MLP to learn nonlinear relationships among pixels. The image is still treated as a vector: nearby pixels do not receive a special shared operation. Dropout is disabled during validation and testing.

## 3.3 Models for spatial and sequential structure

### 3.3.1 Convolutional neural network

The CNN keeps the two-dimensional image layout. Convolution applies the same small filter at different image positions, allowing the network to learn local patterns such as edges and contours [2]. Our model has two convolutional blocks, followed by a dense classifier.

Both convolutions use $3 \times 3$ kernels, stride 1 and padding 1, so they preserve height and width. ReLU follows each convolution. A $2 \times 2$ max-pooling layer then keeps the largest value in each window and halves both spatial dimensions. The first block produces 32 feature maps and the second produces 64.

| Operation | Output shape |
|---|---|
| Input image | $(B, 1, 28, 28)$ |
| $3 \times 3$ convolution, 32 channels; ReLU | $(B, 32, 28, 28)$ |
| $2 \times 2$ max pooling | $(B, 32, 14, 14)$ |
| $3 \times 3$ convolution, 64 channels; ReLU | $(B, 64, 14, 14)$ |
| $2 \times 2$ max pooling | $(B, 64, 7, 7)$ |
| Flatten | $(B, 3136)$ |
| Dense layer, 128 units; ReLU; dropout 0.2 | $(B, 128)$ |
| Dense layer, 10 outputs | $(B, 10)$ |

Table 7: CNN layers and output shapes

The final classifier combines the learned spatial features into ten scores. Although convolution shares weights across positions, the dense layer from 3,136 features to 128 units accounts for most of this model's parameters. The CNN therefore has the largest parameter count in this comparison.

### 3.3.2 LSTM

An LSTM maintains a hidden state and a memory state while reading a sequence [3]. Here, the sequence comes from a static image. The first step receives the top row, the second receives the next row, and the last receives the bottom row. Each step contains 28 pixel values.

For a standardized image $X$, row $t$ is $r_t = X_{0,t,:}$. The batch shape $(B, 28, 28)$ means *batch, row steps, pixel features*. The two dimensions of size 28 have different roles; the sequence is not a sequence of different images.

The implementation uses one forward LSTM layer with 128 hidden units. Its hidden and memory states start at zero for each image. The gates control what information is retained and updated as successive rows are read. After the 28th row, the final hidden state is passed through dropout and a linear classifier. No state is carried from one image to another.

The final hidden state is a summary of all 28 rows. This differs from the CNN, which extracts local features across the two-dimensional image.

## 3.4 Transformer

The Transformer uses attention to combine information from different parts of an image [4]. The input follows the image-patch idea [5], using a small configuration suited to the $28 \times 28$ images.

### 3.4.1 Patch tokens and position information

Each image is divided into a $7 \times 7$ grid of non-overlapping $4 \times 4$ patches. Each patch is flattened into 16 values. Patches are ordered from left to right across each row of the grid, then from top to bottom. A learned linear layer maps each patch from 16 values to 64 features. A learned positional vector is added so the model can distinguish patch locations:

$$e_j = Wp_j + b + q_j, \qquad j = 1, \ldots, 49,$$

where $p_j \in \mathbb{R}^{16}$ is a patch, $W \in \mathbb{R}^{64 \times 16}$ is the projection, and $q_j \in \mathbb{R}^{64}$ is its position vector. Dropout with probability 0.2 is applied after this addition.

### 3.4.2 Attention blocks and classification

Two encoder blocks process the 49 tokens. Each block uses four attention heads, a feature width of 64, and a feedforward network with hidden width 128 and GELU activation. Layer normalization is applied before each attention and feedforward sublayer, with residual connections around both. Dropout is 0.2.

For one attention head, the operation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{16}}\right)V.$$

The queries $Q$, keys $K$ and values $V$ are learned projections of the tokens. Each head has 16 features because the 64 features are divided among four heads. Attention weights determine how much information one patch receives from the others. All 49 patches are available to every patch; no causal mask is used.

| Stage | Shape |
|---|---|
| Flattened image patches | $(B, 49, 16)$ |
| Patch projection and learned positions | $(B, 49, 64)$ |
| Two encoder blocks | $(B, 49, 64)$ |
| Final layer normalization; mean over patches | $(B, 64)$ |
| Dropout 0.2; dense classifier | $(B, 10)$ |

Table 8: Transformer layer dimensions

The final representation is the mean of the 49 normalized output tokens. There is no extra classification token. The encoder internally uses sequence-first tensors of shape $(49, B, 64)$; this transpose changes the layout, not the patch order. All weights are trained from scratch.

## 3.5 Learning objective

Every model learns to assign a high score to the correct class. For image $i$, softmax converts the logits $z_{i,k}$ into class probabilities:

$$p_{i,k} = \frac{\exp(z_{i,k})}{\sum_{j=0}^{9}\exp(z_{i,j})}, \qquad \mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\log p_{i,y_i}.$$

Here, $y_i$ is the correct class and $\mathcal{L}$ is the average cross-entropy loss. The implementation passes logits directly to `CrossEntropyLoss`, which performs the required calculation. Prediction uses $\hat{y}_i = \arg\max_k z_{i,k}$. Class weights are not used because the training split contains the same number of examples from each class.

## 3.6 Training procedure

All five models use Adam [6] with learning rate 0.001, batch size 128 and a budget of ten epochs. These settings give each model the same number of passes over the training data. Table 9 records the shared choices.

| Setting | Value |
|---|---|
| Random seed | 42 for Python, NumPy, PyTorch and the training shuffle |
| Optimizer | Adam, learning rate 0.001 |
| Batch size | 128; the smaller last batch is kept |
| Training duration | 10 complete epochs |
| Loss | Unweighted cross-entropy |
| Regularization | Dropout 0.2 in MLP, CNN, LSTM and Transformer |
| Checkpoint selection | Lowest validation loss; an exact tie keeps earlier weights |
| Data loading | Training shuffled; validation and test ordered; zero workers |
| Other settings | No augmentation, weight decay, gradient clipping or scheduler |

Table 9: Training configuration

Each model is initialized separately after resetting the seed. A training epoch visits all 54,000 training images. For each batch, the program clears the previous gradients, calculates the logits and loss, backpropagates, and updates the weights with Adam.

At the end of the epoch, the model is evaluated on all 6,000 validation images. Evaluation mode disables dropout, and gradient recording is turned off. The program saves the weights whenever validation loss improves. Training continues through epoch ten, even if the best checkpoint was found earlier. The saved checkpoint with the lowest validation loss is then reloaded for testing.

Epoch loss is weighted by the number of images in each batch, so a smaller final batch does not receive the same weight as a full batch. Accuracy and macro-F1 are calculated from predictions across the complete split. Training metrics are collected while the weights change; validation metrics use the fixed weights at the end of an epoch.

## 3.7 Evaluation measures

The selected checkpoint is evaluated on all 10,000 official test images. Accuracy measures the fraction classified correctly:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}_i = y_i).$$

Here, $N$ is the number of evaluated images and $\mathbf{1}$ is one when the prediction is correct, otherwise zero.

Macro-F1 gives equal weight to every class. For class $k$, true positives $TP_k$ are correct predictions of that class, false positives $FP_k$ are other classes predicted as $k$, and false negatives $FN_k$ are examples of $k$ predicted as another class:

$$F1_k = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad \text{MacroF1} = \frac{1}{10}\sum_{k=0}^{9} F1_k.$$

Undefined class scores are assigned zero. Both accuracy and macro-F1 are reported because overall accuracy alone does not show how performance varies among categories.

The evaluation also saves cross-entropy loss, per-class precision and recall, and a confusion matrix. Matrix rows represent true labels and columns represent predicted labels. Correct and incorrect example images help explain recurring mistakes. These outputs support the results and error analysis; they do not select the training checkpoint.

## 3.8 Runtime, reproducibility and comparison scope

### 3.8.1 Runtime measurements

Training time covers the ten-epoch training and validation loop, including saving improved checkpoints. It excludes data preparation before the loop and the final checkpoint reload and validation after it.

Forward-pass timing uses a fixed batch of 128 images already on the device. After ten warm-up calls, the program times 100 forward calls and averages their duration. Dividing the batch time by 128 gives time per image at that batch size. Device synchronization is performed around the timed region. This measurement excludes data loading and host-to-device transfers and describes batched throughput, not the latency of a single-image request.

A separate timer measures the complete test pass, including loading, transfers, model execution, loss and prediction collection. It excludes later metric summaries and plots. The two measurements are reported separately because they cover different work.

### 3.8.2 Reproducibility

The implementation separates data preparation in `a1/data.py`, model definitions in `a1/models.py`, training in `a1/train.py`, and testing in `a1/evaluate.py`. Configuration files specify the shared training settings. The split cache records the training and validation indices and the normalization constants.

Each run saves the selected checkpoint, epoch history and a summary of its configuration and environment. Evaluation checks the recorded data and split hashes and normalization values before using the checkpoint. It also saves per-image predictions so that the reported scores can be checked again.

The following commands illustrate the CNN workflow from the `A1` folder with the project dependencies installed. The other models use the same entry points and their corresponding configuration files.

```
python -m a1.train --model cnn --config configs/cnn.json
python -m a1.evaluate --checkpoint runs/m2/cnn/best.pt --device cuda
```

#### 3.8.3 What the comparison can establish

The split, preprocessing, seed, optimizer, learning rate, batch size and epoch budget are shared. The architecture, input arrangement and parameter count change. The results therefore compare the particular configurations implemented in this assignment, rather than architectures matched by parameter count or FLOPs.

The original Linear and MLP checkpoints were trained on MPS; the remaining models were trained on CUDA. For the runtime comparison, Linear and MLP were also retrained on the GTX 1660 SUPER with the shared settings. Thus, all training times in Table 12 refer to CUDA runs, while the classification tables continue to use the original checkpoints. Each model's reported classification result is based on one seed; repeated seeds would be needed to assess variation in predictive performance.

# 4 Experimental Results

## 4.1 Overall classification performance

Table 10 compares the validation-selected checkpoints. Validation accuracy describes the 6,000-image selection partition; test accuracy describes the separate 10,000-image official test partition. All test metrics refer to the same ordered images. Accuracy is reported as a percentage, while macro-F1 uses a 0–1 scale.

| Model | Epoch | Val. acc. | Test acc. | Test F1 | Test loss | Errors |
|---|---|---|---|---|---|---|
| Linear | 9 | 86.48 | 84.26 | 0.8417 | 0.4454 | 1,574 |
| MLP | 8 | 89.87 | 88.50 | 0.8846 | 0.3251 | 1,150 |
| CNN | 10 | 93.28 | 91.94 | 0.9189 | 0.2366 | 806 |
| LSTM | 9 | 89.80 | 88.48 | 0.8835 | 0.3254 | 1,152 |
| Transformer | 10 | 86.43 | 85.55 | 0.8524 | 0.4085 | 1,445 |

Table 10: Validation and test results

The CNN performs best on the test set in both accuracy and macro-F1, and also has the lowest cross-entropy loss. Its 91.94% accuracy is 3.44 percentage points above the MLP and 7.68 points above the Linear model. Relative to the MLP, it reduces the number of mistakes from 1,150 to 806, a reduction of 344 images or 29.9% of the MLP's errors.

The MLP improves over Linear by 4.24 percentage points. Adding nonlinear hidden layers is therefore useful in this experiment, even though both models use the same flattened input. The CNN improves further while retaining spatial neighborhoods, but its larger capacity also changes the comparison.

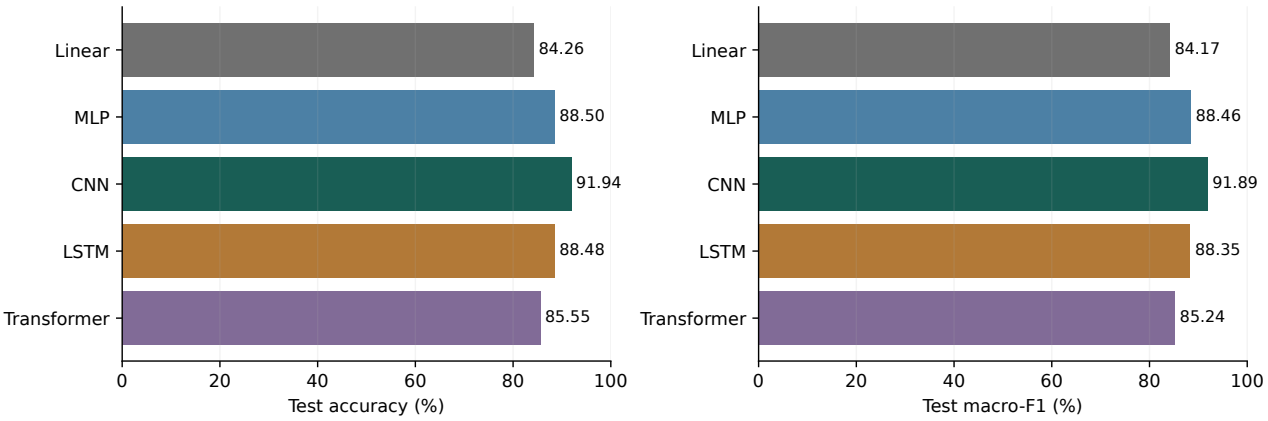



Figure 9: Test accuracy and macro-F1

MLP and LSTM are almost tied: their test accuracies differ by 0.02 percentage points, or two images. This is too small to support a stable ranking without repeated runs. Their macro-F1 values are also close, although their class-level mistakes differ.

The Transformer obtains 85.55% test accuracy, 1.29 percentage points above Linear and below the other non-linear models. This is the result for our two-block patch model after ten epochs; it should not be interpreted as a general comparison between CNNs and Transformers.

Because the test set has equal class support, weighted F1 and macro-F1 are identical here. Accuracy and macro-F1 remain different measures: F1 also depends on how false positives are distributed across classes.

## 4.2 Learning behavior and checkpoint selection

Figure 10 shows the recorded training and validation losses for all ten epochs. The same vertical scale is used for every model. Dotted lines indicate the selected checkpoint.

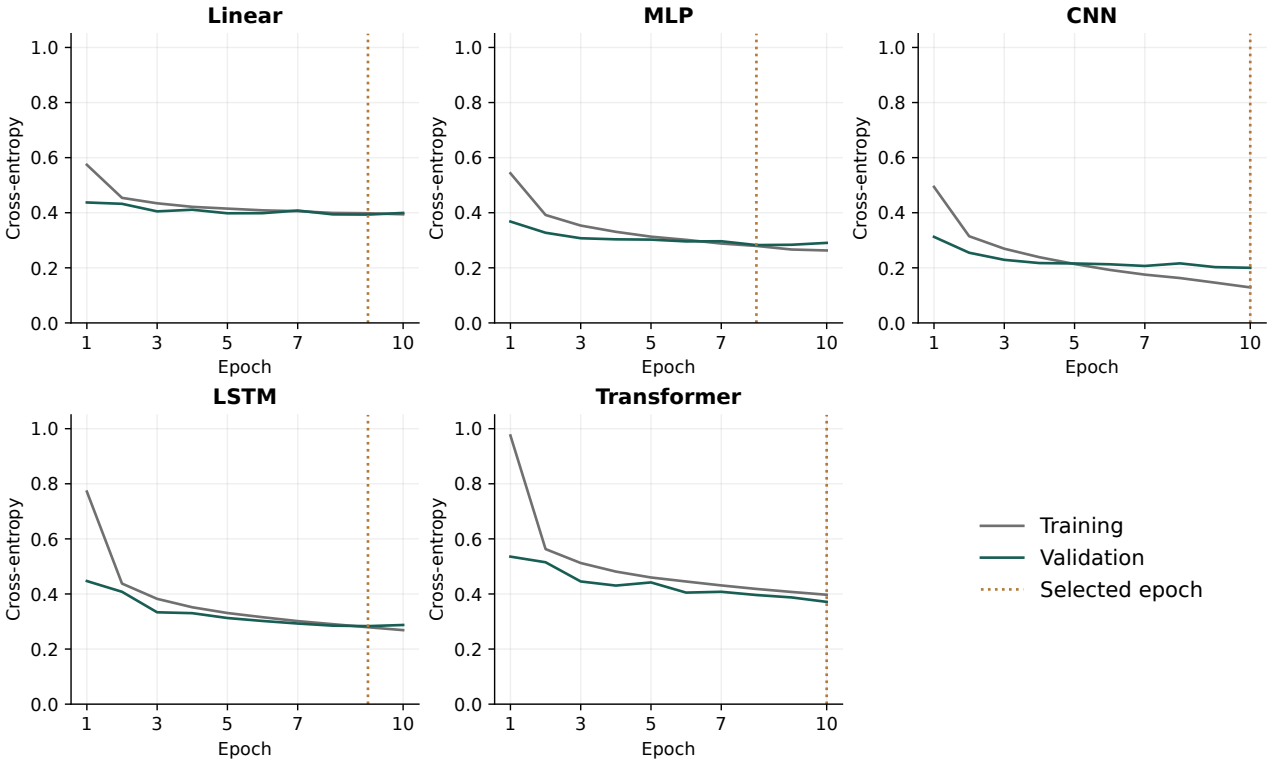



Figure 10: Training and validation loss

| Model | Val. loss, epoch 1 | Val. loss, epoch 10 | Train acc. (%) | Val. acc. (%) |
|---|---|---|---|---|
| Linear | 0.4373 | 0.3993 | 86.17 | 86.15 |
| MLP | 0.3680 | 0.2906 | 90.19 | 89.65 |
| CNN | 0.3124 | 0.2001 | 95.19 | 93.28 |
| LSTM | 0.4469 | 0.2874 | 90.15 | 89.40 |
| Transformer | 0.5355 | 0.3713 | 85.47 | 86.43 |

Table 11: Validation loss and final accuracy

All five models reduce validation loss between the first and last epochs. Linear changes relatively little after its early improvement and selects epoch 9. MLP selects epoch 8: its training loss continues to decrease, but its validation loss rises from 0.2825 at epoch 8 to 0.2906 at epoch 10. LSTM selects epoch 9 and also shows a small validation-loss increase at epoch 10. These patterns support keeping the best checkpoint rather than automatically using the last one.

CNN and Transformer select epoch 10. The Transformer is still improving at the budget boundary, so the available run cannot show whether a longer schedule would close the gap. For CNN, the final training accuracy is 95.19% and validation accuracy is 93.28%, indicating a gap on these recorded metrics.

Validation loss can be lower than training loss, especially early in a run. Dropout is disabled during validation, and validation uses weights after the epoch rather than the changing weights seen throughout training. The two curves do not represent precisely the same evaluation conditions.

## 4.3 Model size and computational cost

| Model | Parameters | Train (s) | Train device | Forward (ms) | Test pass (s) |
|---|---|---|---|---|---|
| Linear | 7,850 | 90.85 | CUDA | 0.060 | 1.110 |
| MLP | 235,146 | 96.69 | CUDA | 0.326 | 1.410 |
| CNN | 421,642 | 105.03 | CUDA | 1.074 | 1.382 |
| LSTM | 82,186 | 122.12 | CUDA | 1.189 | 1.237 |
| Transformer | 71,946 | 139.45 | CUDA | 2.061 | 1.595 |

Table 12: Model size and runtime

Linear is the smallest model, with 7,850 parameters, and has the shortest recorded forward time at 0.060 milliseconds per batch. MLP uses 235,146 parameters and takes 0.326 milliseconds per batch. CNN has 421,642 parameters and takes 1.074 milliseconds per batch. Its accuracy gain therefore comes with a larger model and higher forward cost than these baselines.

LSTM and Transformer are smaller than the MLP, with 82,186 and 71,946 parameters, but their recorded forward times are longer. Parameter count alone does not describe execution cost. The LSTM processes 28 recurrent steps, and the Transformer applies attention and feedforward operations to 49 tokens. Kernel implementation and the chosen batch size also affect runtime.

### 4.3.1 Interpreting training time

All five training times in Table 12 were measured on the GTX 1660 SUPER using CUDA. Linear and MLP were retrained with seed 42, the same 54,000/6,000 split, normalization, batch size 128, Adam learning rate 0.001 and ten-epoch budget. Their CUDA checkpoints and timing records are saved under `runs/cuda_baselines`. The original baseline checkpoints are still used for the classification results and error analysis; the new runs provide comparable-device timing measurements only.

The recorded training times range from 90.85 seconds for Linear to 139.45 seconds for Transformer. Each is an observed ten-epoch duration, not an average across repeated trials. The runs took place in separate sessions, so system load and GPU clock variation may also affect the timings.

### 4.3.2 Interpreting inference time

All five saved inference measurements use the GTX 1660 SUPER, batch size 128, ten warm-up calls and 100 timed forward calls. The data is already on the GPU. The reported batch time excludes loading and host-to-device transfers. For CNN, 1.074 milliseconds per batch corresponds to approximately 0.00839 milliseconds per image *within that batch*. It is not the latency of a single-image request.

The complete test pass takes between 1.110 and 1.595 seconds across the models. It includes additional work from the evaluation pipeline. Because the forward and test-pass timers cover different operations, their difference cannot be used to estimate an exact loading overhead.

The saved summaries do not contain model-specific peak GPU memory or energy measurements. Parameter counts are reported directly, without treating them as a substitute for those missing resource measurements.

# 5 Error Analysis

## 5.1 Performance by class

The overall scores conceal substantial variation among categories. Figure 11 compares the per-class F1 values of all five models. The full numerical table is included in Appendix A.

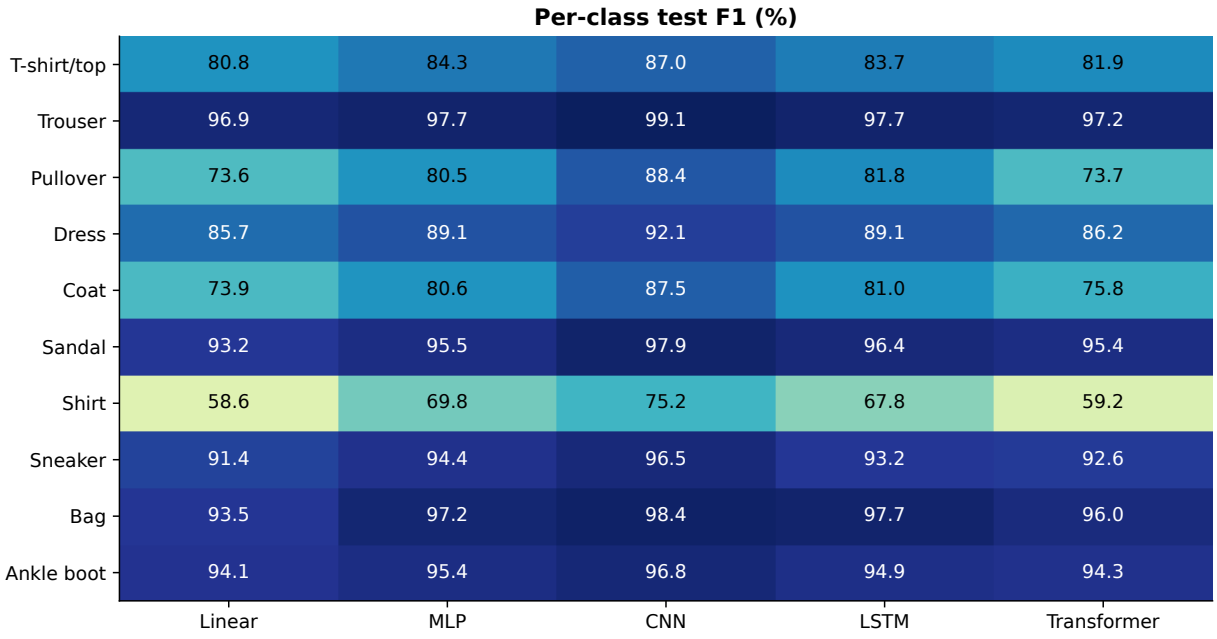



Figure 11: Test F1 by class (%)

Shirt has the lowest F1 for every model. Its score rises from 58.59% for Linear to 69.76% for MLP and 75.18% for CNN. Transformer reaches 59.20%, which is close to Linear on this class despite its higher overall score.

CNN has the highest F1 for each of the ten classes in the recorded runs. Its improvement over MLP is particularly clear for Pullover (80.54% to 88.43%) and Coat (80.65% to 87.45%). For Trouser, both models already perform well, so the gain is smaller.

The LSTM's aggregate result is close to the MLP's, but it has higher F1 on Pullover, Coat, Sandal and Bag, and lower F1 on Shirt and Sneaker. For example, LSTM obtains 96.40% F1 on Sandal versus 95.50% for MLP, but 67.84% on Shirt versus 69.76%. The near tie in accuracy therefore does not mean the two models make the same decisions.

These results broadly agree with the visual observations in Section 2: upper-body categories are difficult to separate, while trousers and several accessory or footwear categories are easier. However, class-average similarity is only a descriptive summary. Models predict individual images, whose shapes and textures can differ substantially from the average.

## 5.2 Confusion patterns in the CNN

Figure 12 shows the CNN's confusion matrix. Rows are true labels and columns are predicted labels. Each row contains 1,000 images.

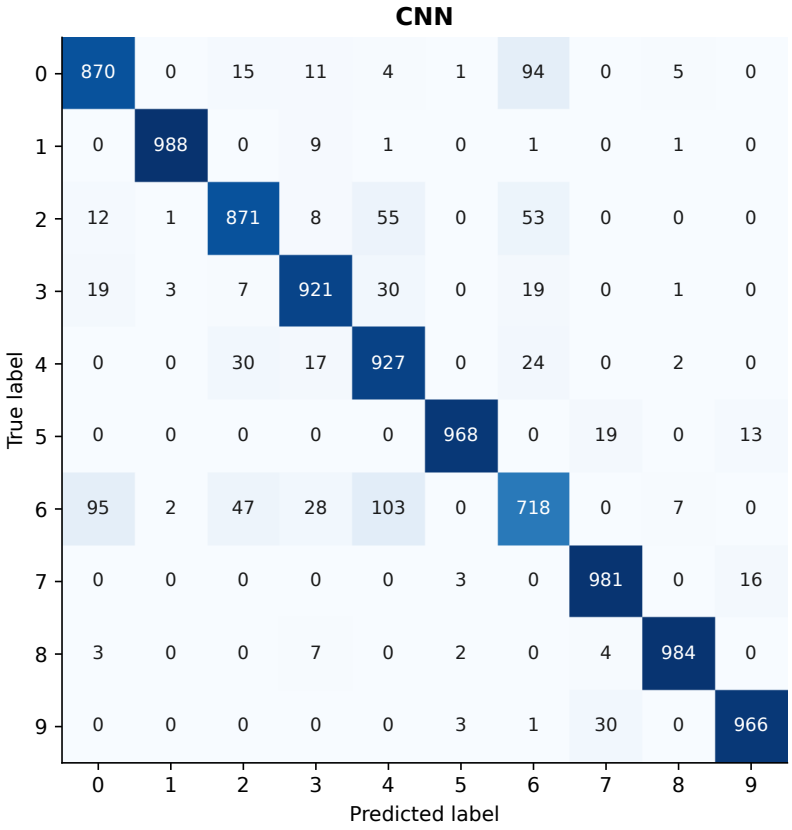



Figure 12: CNN confusion matrix

| Class A | Class B | A predicted as B | B predicted as A |
|---|---|---|---|
| T-shirt/top | Shirt | 94 | 95 |
| Coat | Shirt | 24 | 103 |
| Pullover | Shirt | 53 | 47 |
| Pullover | Coat | 55 | 30 |
| Dress | Shirt | 19 | 28 |
| Dress | Coat | 30 | 17 |

Table 13: Main CNN class confusions

Of the CNN's 806 mistakes, 532 occur between the four labels T-shirt/top, Pullover, Coat and Shirt. These account for 66.0% of all errors. Shirt alone has 282 incorrect predictions: 103 become Coat, 95 become T-shirt/top and 47 become Pullover.

The confusions are not always symmetric. There are 103 Shirt-to-Coat errors but only 24 Coat-to-Shirt errors. CNN recalls 92.7% of Coat images, yet its Coat precision is lower, at 82.77%, because other garments are also predicted as Coat. This distinction explains why recall alone would give an incomplete view of the class.

## 5.3 Individual predictions and shared errors

Figure 13 displays four correct and four incorrect CNN predictions. A seeded generator selects from the correct set and then the incorrect set. The deliberately balanced display illustrates behavior; it does not represent the model's actual error frequency.

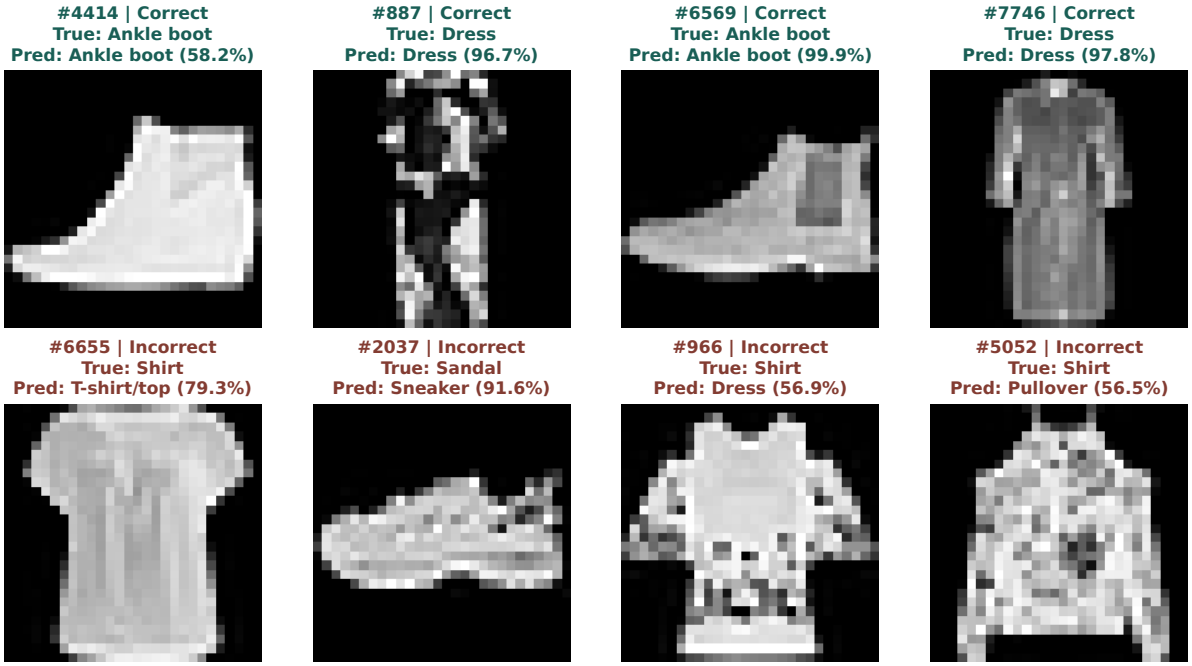



Figure 13: CNN prediction examples

Image #6655 has a short-sleeved outline and is labeled Shirt, but is predicted as T-shirt/top. Image #5052 is also Shirt and is predicted as Pullover. These examples illustrate the upper-body confusion counted in the matrix. The images alone cannot reveal the model's internal reason for each decision.

The Sandal image #2037 is predicted as Sneaker with a score of 91.6%. In contrast, the correctly classified Ankle boot #4414 has a score of 58.2%. High confidence can accompany an error, and moderate confidence can accompany a correct prediction. The study does not include a calibration experiment.

| Compared with CNN | CNN only correct | Other only correct | Both wrong |
|---|---|---|---|
| Linear | 998 | 230 | 576 |
| MLP | 566 | 222 | 584 |
| LSTM | 621 | 275 | 531 |
| Transformer | 874 | 235 | 571 |

Table 14: Paired test predictions

CNN corrects 566 images that MLP gets wrong, but MLP correctly handles 222 images that CNN misses. The net difference is the 344-image advantage in Table 10. MLP and LSTM also differ on individual cases: MLP alone is correct on 360 images and LSTM alone on 358, producing their two-image net gap. All five models are wrong on 389 shared images.

These complementary decisions motivate future ensemble experiments, but no ensemble has been trained or evaluated here. The paired counts describe agreement and disagreement; they are not an achieved ensemble score.

# 6 Discussion and Limitations

## 6.1 What the comparison shows

The strongest result comes from the CNN. Its local filters and pooling provide a natural way to process the spatial structure of clothing images. The class-level results show gains across all categories, including the similar

upper-body garments that trouble the baselines. This is consistent with the usefulness of local spatial features in this task.

That explanation remains an interpretation. The CNN also has the largest parameter count, and no ablation isolates the contribution of convolution, pooling, width or regularization. The experiment demonstrates a better result for this configuration, rather than proving a single cause for the improvement.

The MLP is a useful practical baseline. It improves substantially over Linear and nearly matches the LSTM while requiring less recorded forward time. The LSTM achieves similar overall accuracy with fewer parameters, but reading the image sequentially does not provide an accuracy advantage in this run. Its final hidden state must summarize the rows before classification.

The Transformer has the fewest parameters among the four nonlinear models, but it has the longest recorded forward time and a lower test score than MLP, CNN or LSTM. Attention is not automatically an improvement under a small, fixed training configuration. Since its validation loss is still decreasing at epoch 10, longer training is a plausible follow-up, not an established remedy.

## 6.2 Limitations

The experiment uses a single training seed for each model. We can describe the observed differences, but cannot estimate variation across different model initializations. This matters especially for the MLP and LSTM, whose test accuracies are separated by only two images.

Although the data split, optimizer and number of epochs are fixed, model capacities differ considerably. The CNN has more than 400,000 parameters, while the Linear baseline has fewer than 8,000. The results do not isolate architecture from capacity, nor does the shared ten-epoch budget guarantee that every model has converged. We did not run a systematic hyperparameter search.

All reported training times now use the same GTX 1660 SUPER, although Linear and MLP were timed in separate CUDA retraining runs rather than the original runs used for accuracy. The sessions were not repeated under identical system load, so these measurements show observed cost rather than a stable speed ranking. Inference is measured at batch size 128 only, and peak GPU memory use is not available from the saved runs.

Finally, Fashion-MNIST contains small grayscale product images on simple backgrounds. Performance on photographs with color, background clutter or occlusion remains untested. The error analysis was carried out after test evaluation and was not used to select new model settings.

## 6.3 Next experiments

The first priority is to repeat each configuration across several seeds and report the mean and variation of accuracy and macro-F1. Runtime measurements should also be repeated on the same GPU under a consistent timing setup.

Next, use validation data to compare longer schedules and modest learning-rate changes, especially for Transformer. Separate ablations can examine CNN width, LSTM readout and patch size while controlling one factor at a time. Augmentation and calibration are further candidates. Any new settings should be selected through validation before a clearly documented final evaluation.

# 7 Conclusion

This assignment examined five image classifiers using the same Fashion-MNIST split, normalization and ten-epoch training budget. Each model was evaluated using the checkpoint with the lowest validation loss.

Among the tested configurations, the CNN gives the best result, reaching 91.94% test accuracy and 0.9189 macro-F1. It makes 806 mistakes out of 10,000 test images, compared with 1,150 for the MLP. Upper-body garments, particularly Shirt, remain the hardest classes to distinguish. The MLP and LSTM have almost identical overall accuracy (88.50% and 88.48%), although they disagree on many individual predictions.

The MLP also has a considerably shorter measured forward time than the CNN, so it remains a reasonable choice where computational cost matters. The Transformer reaches 85.55% in this experiment; its validation loss was still improving at the final epoch, making longer training worth investigating.

These findings apply to the configurations and single runs reported here. Repeated seeds, controlled changes to model size and repeated runtime measurements would be needed for a stronger comparison.

# References

[1] Han Xiao, Kashif Rasul, and Roland Vollgraf. "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms". In: (2017). arXiv: 1708.07747.

[2] Yann LeCun et al. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[3] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[4] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. 2017.

[5] Alexey Dosovitskiy et al. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations*. 2021.

[6] Diederik Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations*. 2015.

# A Detailed Class Results

| Class | Linear | MLP | CNN | LSTM | Transformer |
|---|---|---|---|---|---|
| T-shirt/top | 80.81 | 84.27 | 87.04 | 83.66 | 81.91 |
| Trouser | 96.86 | 97.67 | 99.10 | 97.73 | 97.20 |
| Pullover | 73.58 | 80.54 | 88.43 | 81.84 | 73.74 |
| Dress | 85.69 | 89.14 | 92.05 | 89.14 | 86.20 |
| Coat | 73.93 | 80.65 | 87.45 | 81.04 | 75.83 |
| Sandal | 93.24 | 95.50 | 97.93 | 96.40 | 95.36 |
| Shirt | 58.59 | 69.76 | 75.18 | 67.84 | 59.20 |
| Sneaker | 91.38 | 94.44 | 96.46 | 93.17 | 92.59 |
| Bag | 93.54 | 97.24 | 98.40 | 97.73 | 96.00 |
| Ankle boot | 94.08 | 95.38 | 96.84 | 94.95 | 94.34 |

Table 15: Per-class test F1 (%)

| Class | Precision | Recall | F1 | Errors |
|---|---|---|---|---|
| T-shirt/top | 87.09 | 87.00 | 87.04 | 130 |
| Trouser | 99.40 | 98.80 | 99.10 | 12 |
| Pullover | 89.79 | 87.10 | 88.43 | 129 |
| Dress | 92.01 | 92.10 | 92.05 | 79 |
| Coat | 82.77 | 92.70 | 87.45 | 73 |
| Sandal | 99.08 | 96.80 | 97.93 | 32 |
| Shirt | 78.90 | 71.80 | 75.18 | 282 |
| Sneaker | 94.87 | 98.10 | 96.46 | 19 |
| Bag | 98.40 | 98.40 | 98.40 | 16 |
| Ankle boot | 97.09 | 96.60 | 96.84 | 34 |

Table 16: CNN per-class metrics

For class $k$, precision is $TP_k/(TP_k + FP_k)$ and recall is $TP_k/(TP_k + FN_k)$. The CNN's Shirt recall is 71.80%, meaning it correctly classifies 718 of the 1,000 Shirt images. Its Shirt precision is 78.90%, meaning that some images predicted as Shirt actually belong to another class.

The exact values are recomputed from the saved confusion counts before rounding. All classes have equal support, so the mean class recall equals overall accuracy. This equality does not make precision or F1 redundant.

# B Additional Confusion Matrices

The matrices below use the same label mapping and color range as the CNN matrix. Rows are true labels and columns are predictions. Each row sums to 1,000 and each complete matrix contains 10,000 predictions.

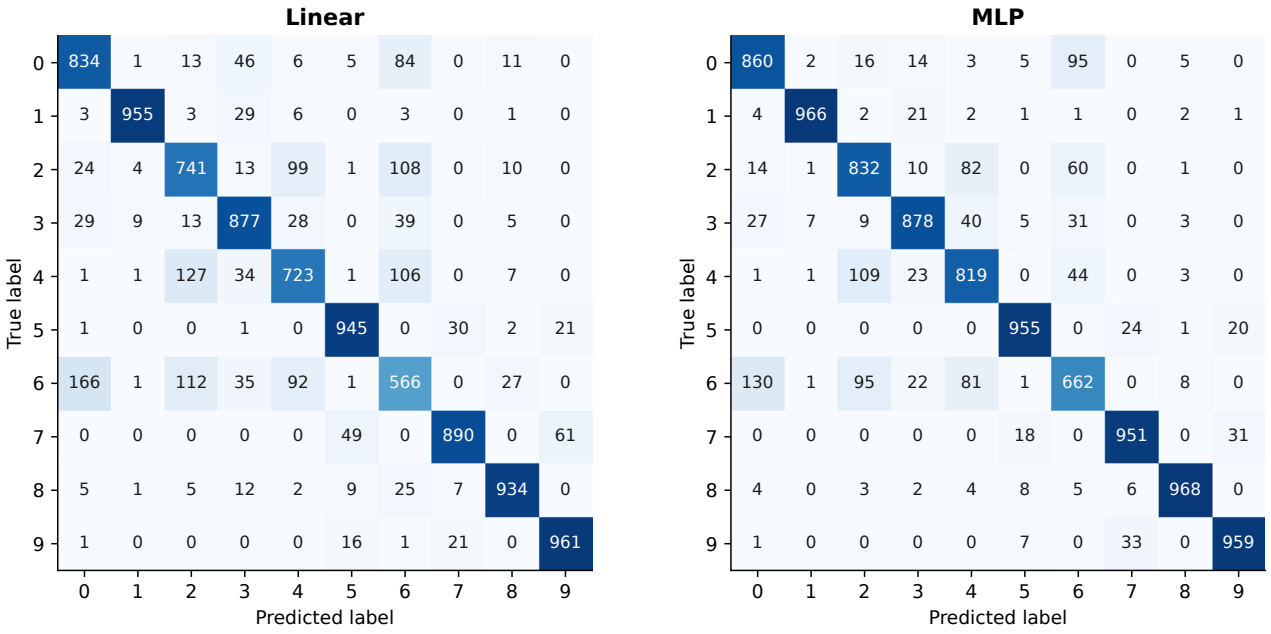



Figure 14: Linear and MLP confusion matrices

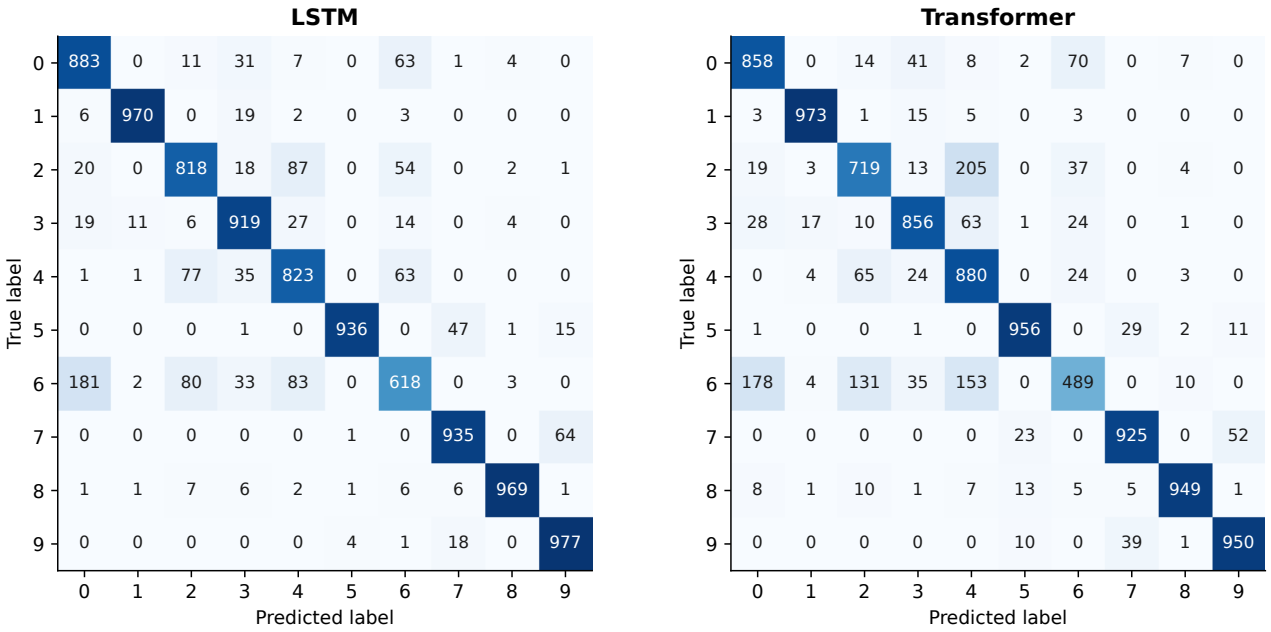



Figure 15: LSTM and Transformer confusion matrices

Labels: 0 T-shirt/top; 1 Trouser; 2 Pullover; 3 Dress; 4 Coat; 5 Sandal; 6 Shirt; 7 Sneaker; 8 Bag; 9 Ankle boot.

# C Reproducibility and Evidence

## C.1 Environment and saved files

The CUDA training and test-evaluation records identify Windows 11, Python 3.12.10, PyTorch 2.9.1+cu128, torchvision 0.24.1+cu128, NumPy 2.2.6 and scikit-learn 1.7.2. The GPU is an NVIDIA GeForce GTX 1660 SUPER with 6 GiB

of memory. The original Linear and MLP checkpoints were trained on MPS and evaluated on CUDA. Separate CUDA training runs for these two models provide the updated runtime measurements.

From the repository's `A1` folder, the principal files are:

| Path | Role |
|---|---|
| `a1/data.py` | Split, normalization and input arrangements. |
| `a1/models.py` | Definitions of all five classifiers. |
| `a1/train.py` | Training loop and checkpoint selection. |
| `a1/evaluate.py` | Test evaluation, predictions and timing. |
| `split_cache.json` | Fixed indices, hashes and normalization values. |
| `runs/m1/linear`, `runs/m1/mlp` | Baseline training and test artifacts. |
| `runs/m2/cnn`, `lstm`, `transformer` | Three model directories under `runs/m2`. |
| `runs/cuda_baselines` | CUDA retraining and timing of Linear and MLP. |
| `report/data/eda_metrics.json` | Training-data statistics and integrity checks. |

Table 17: Experiment files and outputs

Each run directory contains the selected checkpoint (`best.pt`), the epoch history (`history.csv`) and a run summary (`summary.json`). Test outputs include prediction records, confusion counts and a verification file. The files are named `test_summary.json`, `test_predictions.csv`, `confusion_matrix.csv` and `verification.json`.

The test summary records the checkpoint SHA-256 digest, allowing the metrics to be associated with the exact saved file.

## C.2 Commands to reproduce the experiment

Create a Python 3.12 environment and install `requirements-windows.txt` on the CUDA reference platform. For example, run the following from `A1` in PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-windows.txt
$env:CUBLAS_WORKSPACE_CONFIG = ':4096:8'
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
.\.venv\Scripts\python.exe -m a1.train --model cnn --config configs/cnn.json
.\.venv\Scripts\python.exe -m a1.evaluate --checkpoint runs/m2/cnn/best.pt
.\.venv\Scripts\python.exe tests/verify_run.py --model cnn
```

Linear and MLP use `configs/m1.json`; LSTM and Transformer use `configs/lstm.json` and `configs/transformer.json`. Change the model name and checkpoint directory accordingly. The CUDA baseline timing runs are stored in `runs/cuda_baselines`; the original runs under `runs/m1` remain the source of classification metrics. Keep these outputs separate when reproducing the benchmarks. Actual training times may vary with GPU load and clock speed.

## C.3 Checks performed for this report

The report preparation checks all 50,000 saved prediction rows against the local test labels, recomputes accuracy, per-class scores and confusion counts, and verifies the selected epoch against each ten-row history. It also checks checkpoint digests, shared split and source hashes, normalization constants, and the archived EDA file and figure digests.

The existing run-verification files record successful checkpoint reload and CPU/CUDA prediction checks. Those model-execution checks are prior evidence; report preparation does not rerun inference or training. Cross-entropy losses and timings are taken from the saved evaluation summaries, because predicted labels and maximum softmax scores alone cannot reconstruct full test loss. The report's own evidence and document checks are stored in its `qa` folder.